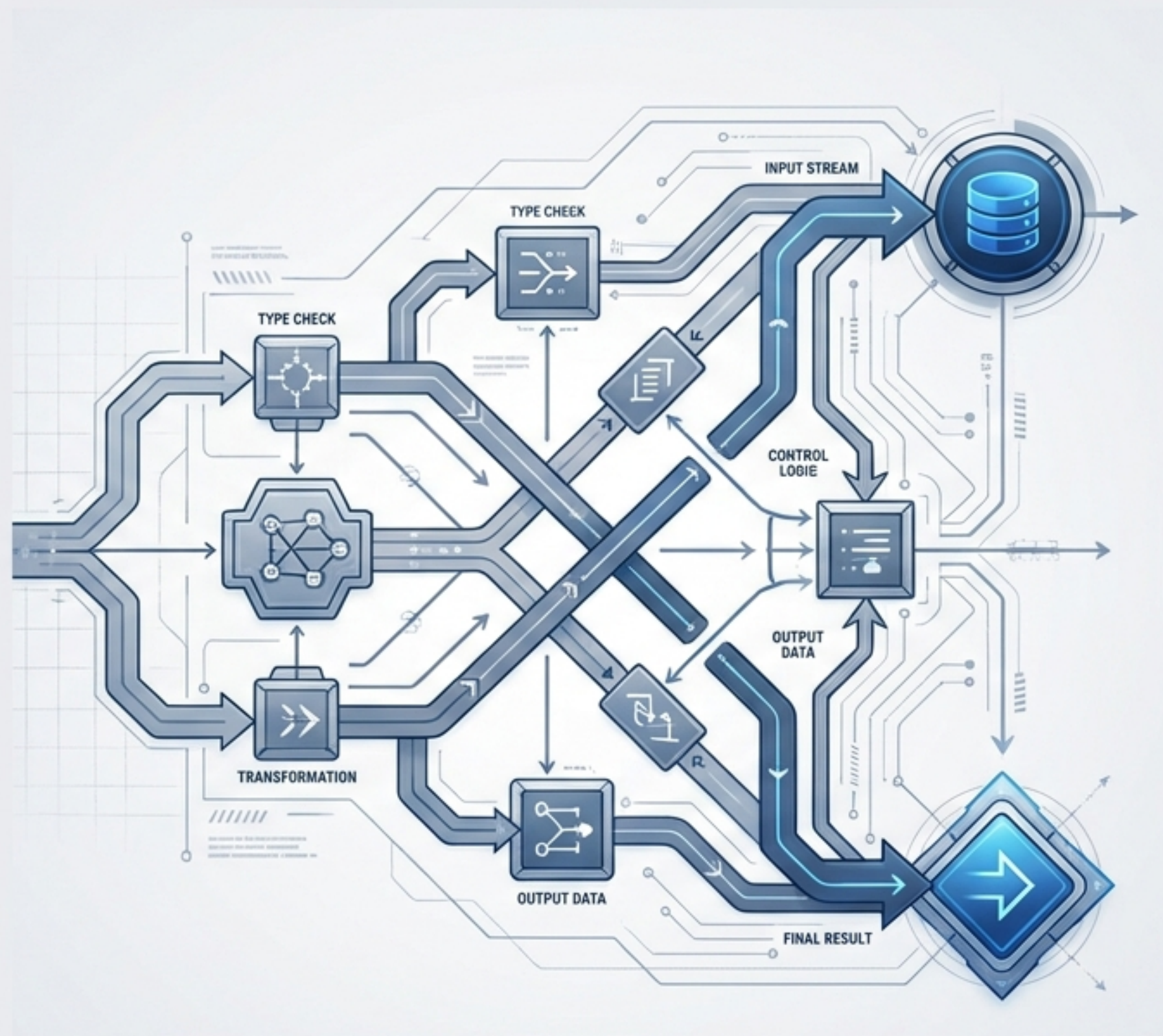


기본 문법 시리즈

데이터의 흐름과 통제

C# 개발자를 위한
타입 제어와 변환 툴킷



내 데이터의 정체를 밝히는 도구

개발 환경 (비주얼 스튜디오) 확인

```
int number = 10;  
Console.WriteLine(number);
```

[local] (지역 변수) int number

런타임 (코드 실행) 확인

GetType() 메서드를 호출하여 변수나 상수의 실제 System 타입을 직접 추출합니다.

```
System.Int32  
System.Int64  
System.Single  
System.Double  
System.Char  
System.String
```

```
Console.WriteLine((273).GetType());
```


수동 타이핑의 종말: var 키워드

컴파일러가 대입되는 값을 분석하여 자료형을 자동으로 추론하고 지정합니다.

The Hard Way

```
int number = 100;  
string text = "Hello";
```

The Smart Way

```
var number = 100;
```

var

 readonly struct System.Int32
Represents a 32-bit signed integer.

var는 타입이 없는 것이 아닙니다. 컴파일러가 대신 타입을 결정해주는 강력한 자동화 도구입니다.

var 사용의 절대 규칙 (Constraints Matrix)

Rule 1: 초기화 - Initialization	Rule 2: 타입 고정 - Strict Typing
<pre>var number;</pre> ⊗ CS0818: 암시적으로 형식화된 지역 변수는 초기화해야 합니다. ! CS0168: 'number' 변수가 선언되었지만 사용되지 않았습니다. 💡	<pre>01 var number = 100; 02 <u>number</u> = "변경";</pre> ⊗ CS0029: 암시적으로 'string' 형식을 'int' 형식으로 변환할 수 없습니다. System.Strin System.String View Election Haniboam (켄) 오속백보기
<pre>var number = 20;</pre> 정상 작동	var는 첫 대입 시점에 타입이 영구적으로 확정됩니다. string을 int로 바꿀 수 없습니다.

var의 생존 공간: 지역 변수 전용

Class (전역 공간 / Global)

```
class Program
```

```
{  
  var global = 52;  
  ...  
}
```

CS0825: 상황별 키워드 'var'는 지역 변수 선언이나 스크립트 코드에만 표시할 수 있습니다.

Method (지역 공간 / Local)



var

```
static void Main(string[] args)  
{  
  var local = 273; ✓  
  ...  
}
```

이유: 클래스 영역은 문맥을 파악하기엔 너무 넓습니다. 메서드 내부에서만 즉시 초기화와 타입 추론이 가능합니다.

컴파일러를 돕는 힌트: 리터럴 접미사 (Suffixes)

숫자만 입력하면 컴파일러는 기본적으로 int나 double로 추론합니다.
다른 타입을 원한다면 명시적 기호가 필요합니다.

코드 선언 (Code Declaration)	추론된 타입 (Inferred Type)
<code>var a = 100L;</code>	➡ <code>long</code>
<code>var b = 100.0;</code>	➡ <code>double</code>
<code>var c = 100.0F;</code>	➡ <code>float</code>

⚠ 소문자 'l'의 함정

65432l처럼 소문자를 사용하면 숫자 1과 혼동됩니다.
비주얼 스튜디오는 CS0078 경고를 발생시킵니다.
항상 대문자 L을 사용하세요.

65432l;

⚠ `readonly struct System.Int64`
Represents a 64-bit signed integer.

⚠ CS0078: 접미사 'l'은 숫자 '1'과 쉽게 혼동됩니다.
쉽게 구별할 수 있도록 'L'을 사용하세요.

외부 세계와의 통로: 입력 (Input)



```
01 static void Main(string[] args)
02 {
03     string input = Console.ReadLine();
04     Console.WriteLine("input: " + input);
05 }
```

string? Console.ReadLine()

표준 입력 스트림에서 다음 줄의 문자를 읽습니다.

ReadLine()의 반환형은 항상 **문자열(string)**입니다.
사용자가 숫자를 입력해도 시스템은 이를 텍스트로 인식합니다.

자료형 변환의 두 가지 철학 (Type Casting Matrix)

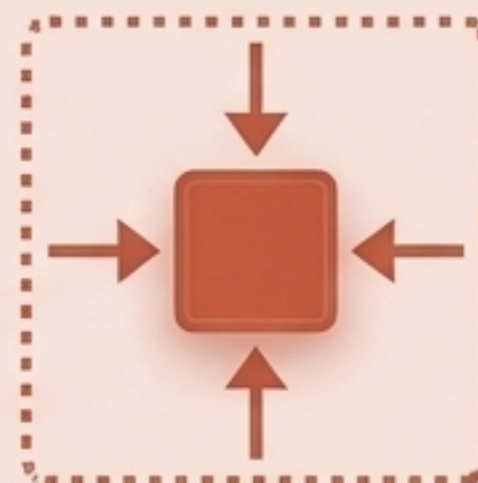
암시적 변환 (Implicit Cast) - 안전한 확장



✓ Safe

- 방향 (Direction): 작은 그릇 ➡ 큰 그릇
- 문법 (Syntax): 추가 코드 불필요 (자동)
- 데이터 손실 (Risk): 없음 (Safe)

명시적 변환 (Explicit Cast) - 강제 압축

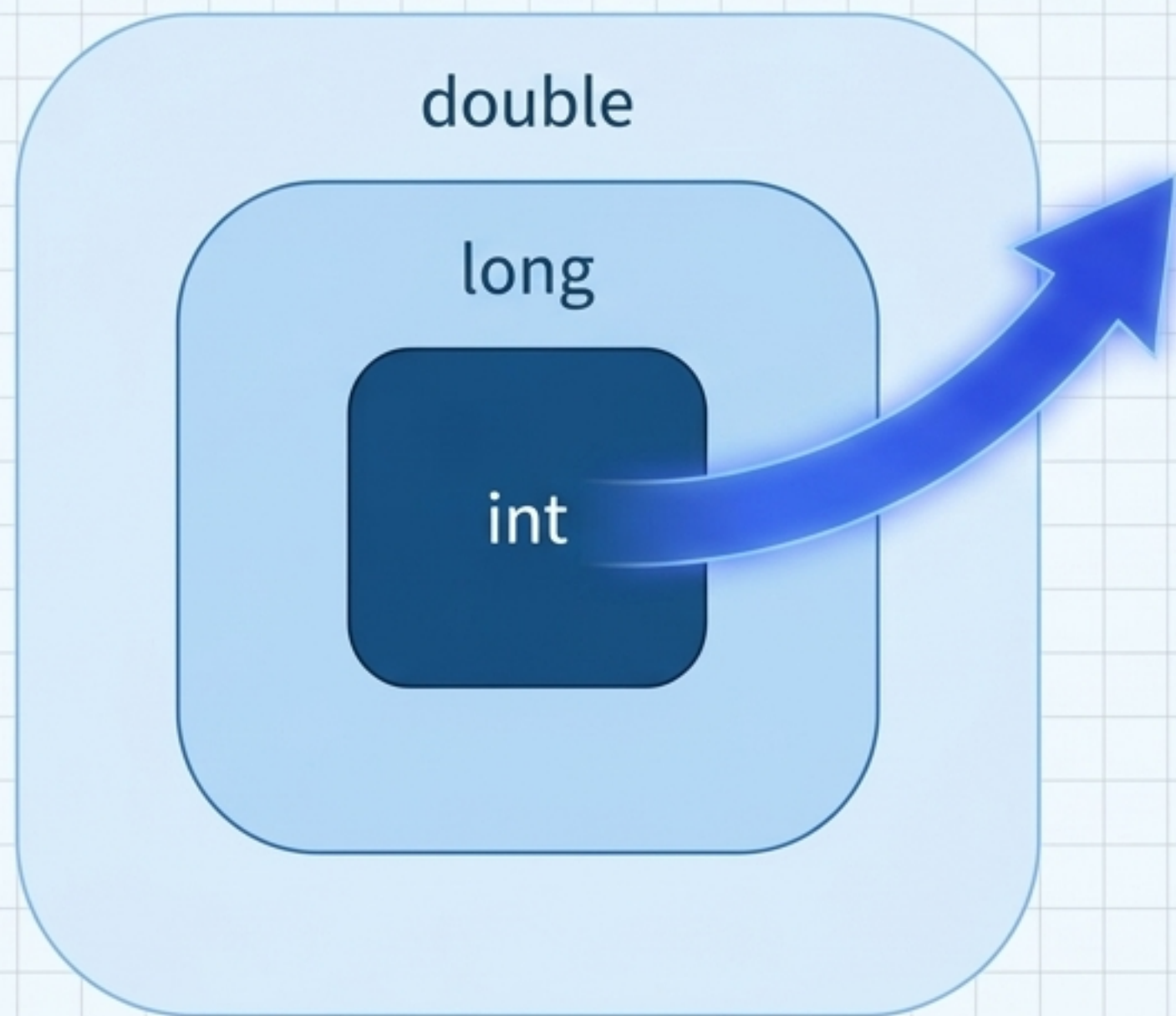


⚠ Warning

- 방향 (Direction): 큰 그릇 ➡ 작은 그릇
- 문법 (Syntax): (type) 캐스트 연산자 필수
- 데이터 손실 (Risk): 발생 가능성 높음 (Warning)

암시적 변환: 자동으로 이루어지는 안전한 확장

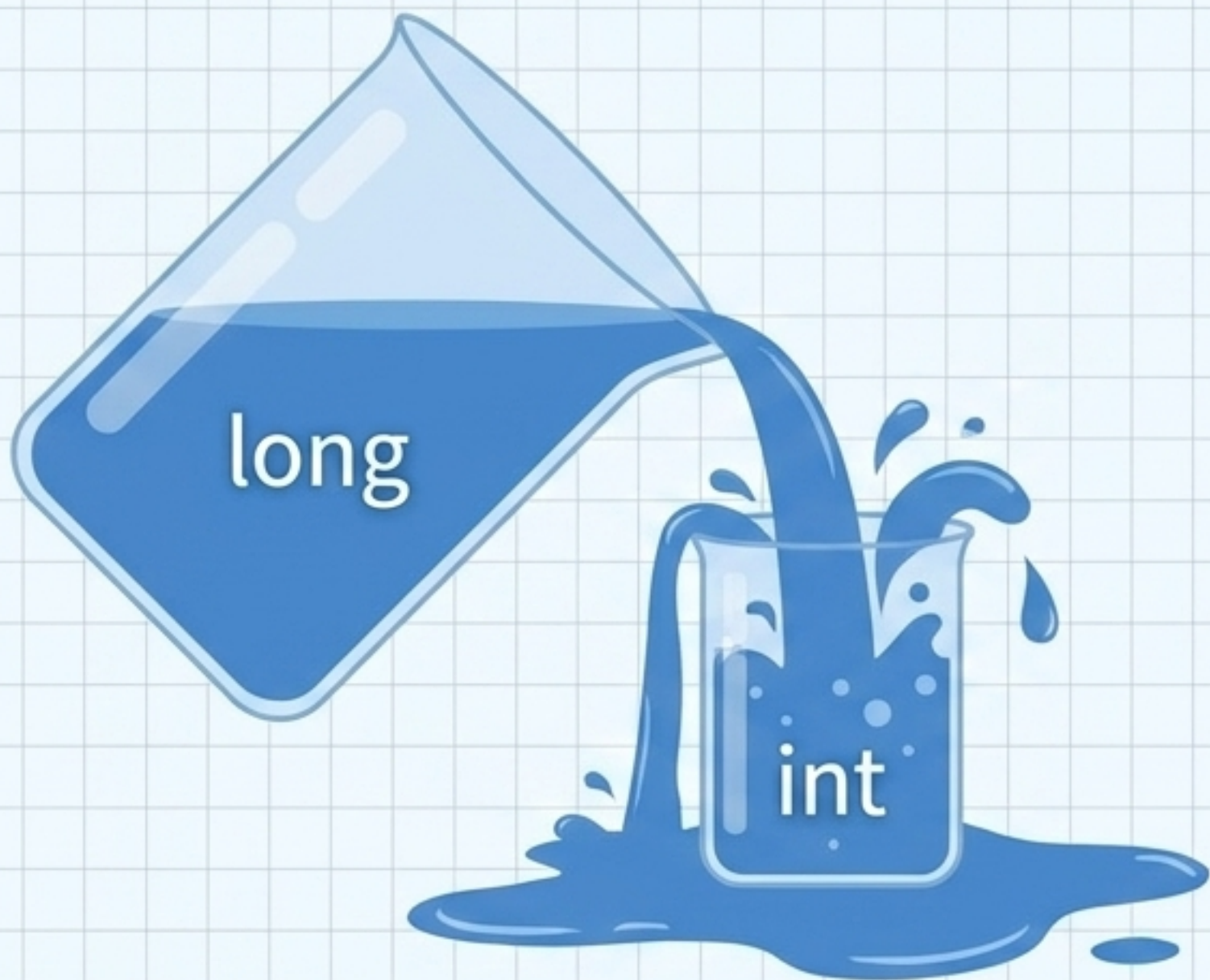
데이터 손실이 일어나지 않는 범위 내에서 C#은 자동으로 자료형을 변환합니다.



```
01 static void Main(string[] args)
02 {
03     // int 자료형의 숫자를 생성합니다.
04     int intNumber = 2147483647;
05
06     // int 자료형을 long 자료형으로 자동 변환합니다.
07     long intToLong = intNumber;
08     Console.WriteLine(intToLong);
09
10     // int 자료형을 double 자료형으로 자동 변환합니다.
11     double intToDouble = intNumber;
12     Console.WriteLine(intToDouble);
13 }
```


명시적 변환과 데이터 오버플로우 (Data Loss)

큰 자료형을 작은 자료형에 강제로 넣을 때, 그릇의 한계를 넘어서면 데이터가 깨집니다.



강제 자료형 변환

```
01 static void Main(string[] args)
02 {
03     // long 자료형을 int 자료형으로 변환합니다.
04     long longNumber = 2147483647L + 2147483647L;
05     int intNumber = (int)longNumber;
06     Console.WriteLine(
07 }
```

(지역 변수) long longNumber

CS0266: 암시적으로 'long' 형식을 'int' 형식으로 변환할 수 없습니다. 명시적 변환시 확인하십시오.

장제적 수정 사항 표시 (Alt+Enter 또는 Ctrl+.)

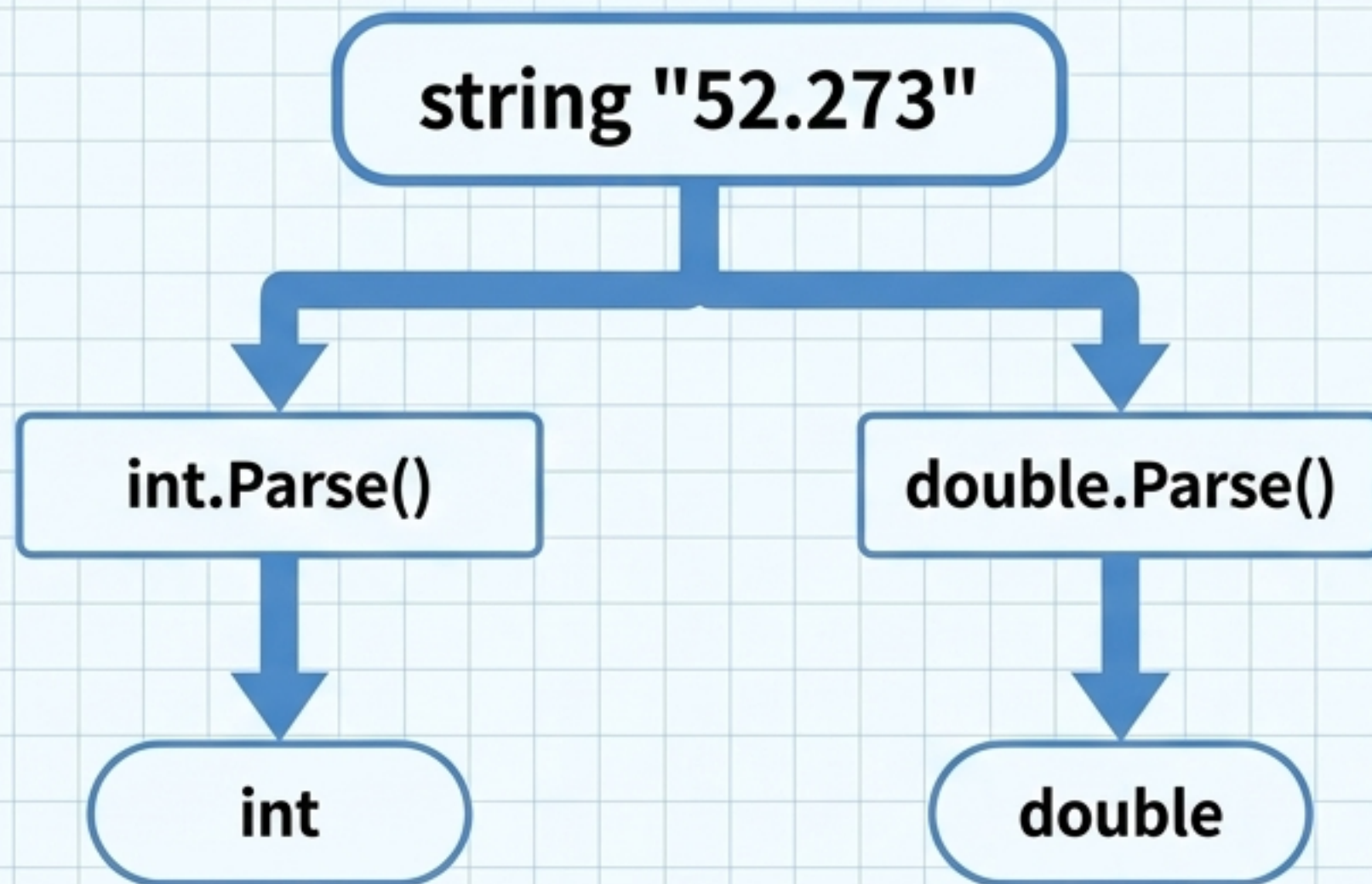
실행 결과

-2

컴파일러 에러를 방지하기 위해 (int)를 쓰지만, 발생하는 이
거대한 오버플로우 에러는 온전히 개발자의 책임입니다.

데이터 번역 허브: 문자열을 숫자로 (Parse)

문자열 데이터를 숫자 형식으로 변환하는 과정을 Parse 메서드를 통해 설명합니다.



```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(int.Parse("52"));
04     Console.WriteLine(long.Parse("273"));
05     Console.WriteLine(float.Parse("52.273"));
06     Console.WriteLine(double.Parse("103.32"));
07
08     Console.WriteLine(int.Parse("52").GetType());
09     Console.WriteLine(long.Parse("273").GetType());
10     Console.WriteLine(float.Parse("52.273").GetType());
11     Console.WriteLine(double.Parse("103.32").GetType());
12 }
```

실행 결과

```
52
273
52.273
103.32
System.Int32
System.Int64
System.Single
System.Double
```

Danger Zone



숫자로 바꿀 수 없는 문자열(abc)을 Parse()에 넣으면 런타임 예외(FormatException)가 발생하며 프로그램이 뺏어버립니다.

```
Console.WriteLine(int.Parse("abc"));
```


데이터 번역 허브: 숫자를 문자열로 (ToString)

ToString Formatting



```

01 static void Main(string[] args)
02 {
03     double number = 52.273103;
04     Console.WriteLine(number.ToString("0.0")); // ➡ 52.3
05     Console.WriteLine(number.ToString("0.00")); // ➡ 52.27
06     Console.WriteLine(number.ToString("0.000")); // ➡ 52.273
07     Console.WriteLine(number.ToString("0.0000")); // ➡ 52.2731
08 }

```

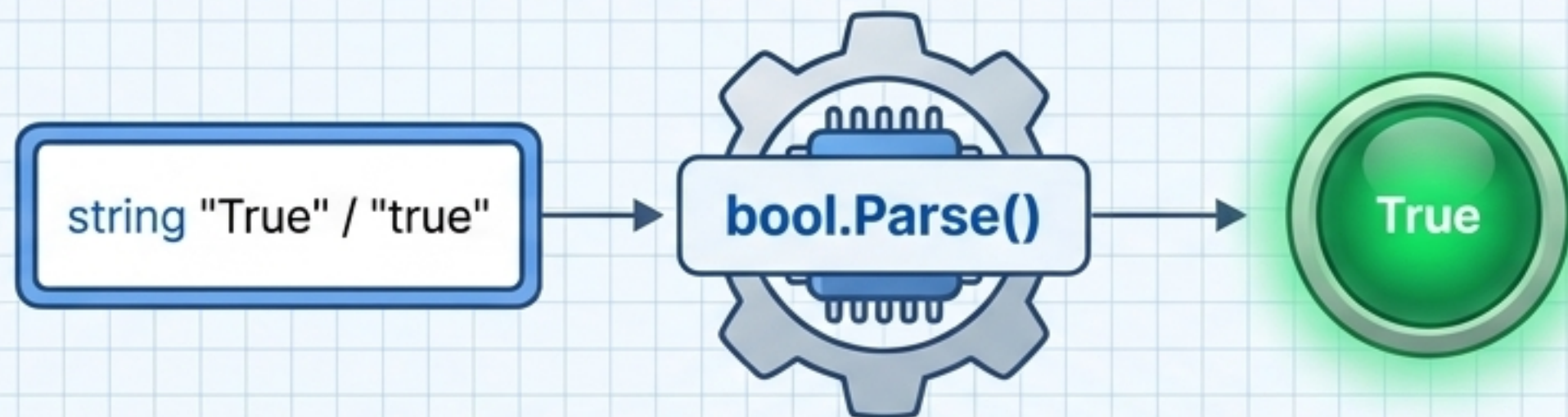
The '+' Operator Context

숫자와 문자열 사이에 +를 쓰면, 숫자는 강제로 문자열로 변환되어 연결(Concatenation)됩니다.

산술 덧셈 (Arithmetic Addition)	문자열 연결 (String Concatenation)
52 + 273 ➡ 325	"52" + 273 ➡ 52273

논리의 번역: 문자열을 불(Bool)로

논리값(True/False)도 문자열에서 추출해낼 수 있습니다.

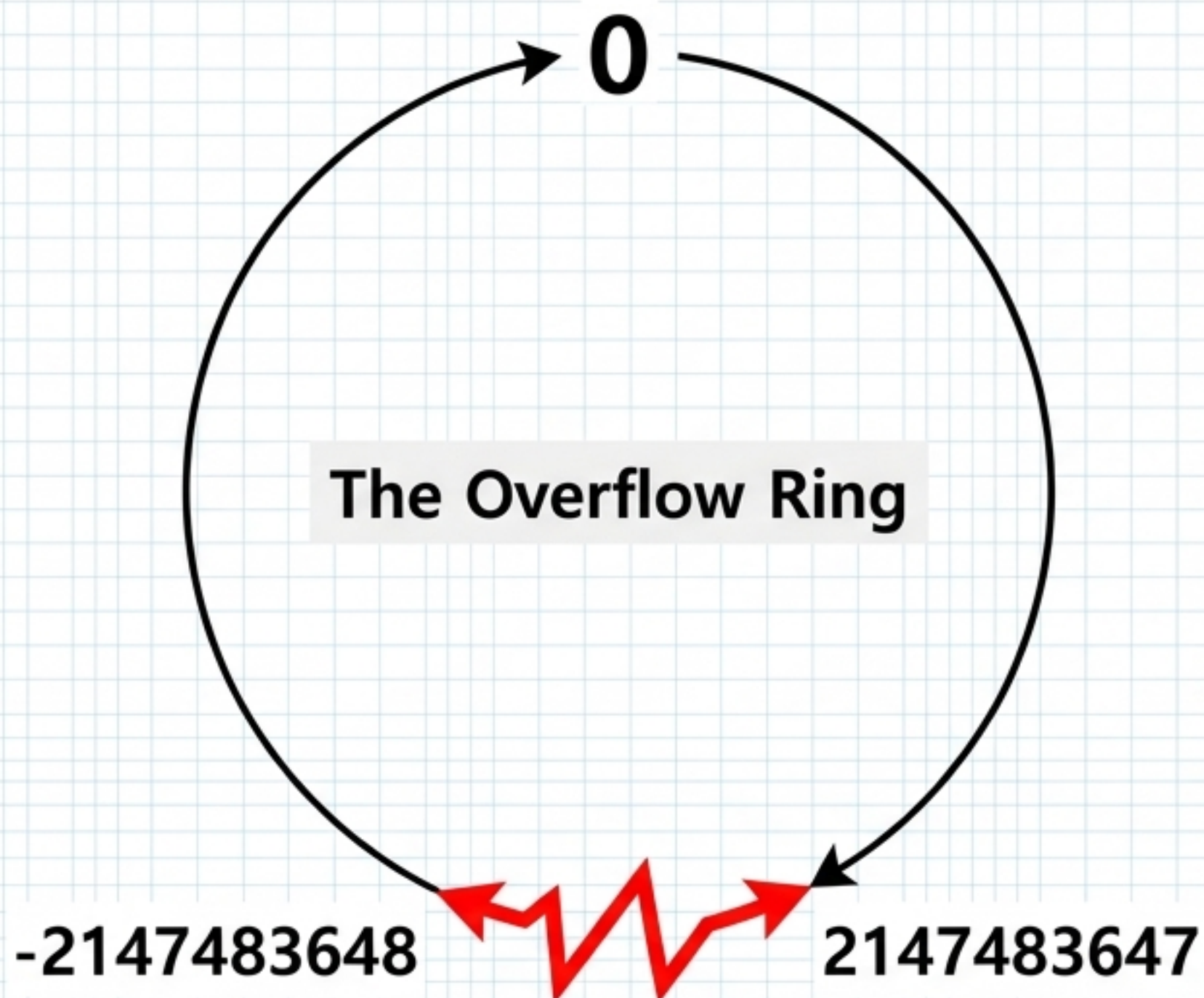


```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(bool.Parse("True"));
04     Console.WriteLine(bool.Parse("true"));
05     Console.WriteLine(bool.Parse("False"));
06     Console.WriteLine(bool.Parse("false"));
07 }
```

실행 결과

```
True
True
False
False
```


인티저 패러독스: 양수가 없는 음수



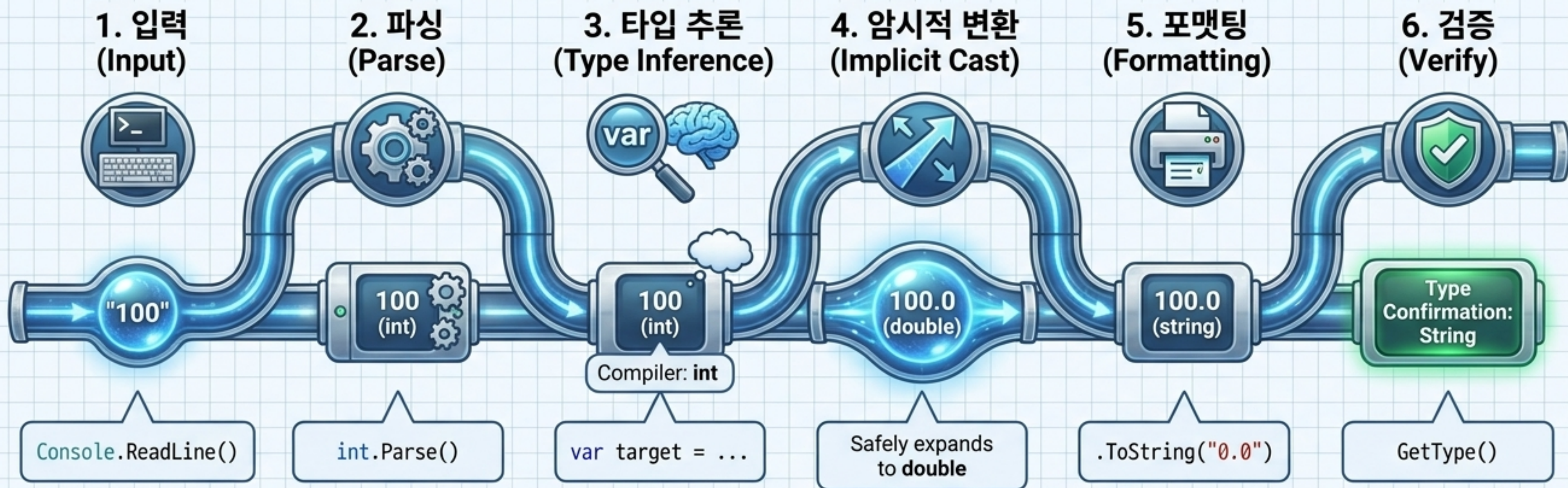
- int의 최솟값은 -2147483648이지만, 최댓값은 2147483647입니다.
- 최솟값에 음수 부호(-)를 붙여 양수로 만들려고 하면 오버플로우가 발생합니다.

```
static void Main(string[] args)
{
    Console.WriteLine(-(-2147483648));
}
```

readonly struct System.Int32
부호 있는 32비트 장수를 나타냅니다.

CS0220: checked 모드에서 잠과일하면 작업이 오버플로됩니다.

마스터 클래스: C# 유니버설 데이터 플로우



모든 데이터는 **명확한 타입(Type)**을 가지며,
우리는 이 흐름과 변환을 완벽하게 통제할 수 있습니다.